Rapport Final

Traitement Avancé du Langage Naturel

Classification et génération juridiques

CLAIR Maël Benjamin Morin

12 mars 2026

Résumé

Ce document présente une synthèse scientifique des trois exercices du TP de TAL, avec un objectif double : (i) produire des benchmarks reproductibles sous contrainte vdigpu et (ii) interpréter les résultats au-delà d'une simple lecture de scores. Le travail couvre la classification de clauses juridiques (LEDGAR), le résumé de documents législatifs (BillSum) et l'explication de clauses pour non-experts (LEDGAR). Chaque configuration est exécutée trois fois avec des graines différentes, et les résultats sont reportés en moyenne et écart-type. La contribution principale du rapport est de montrer comment la performance dépend de l'alignement entre architecture, formulation de tâche et contraintes système, en particulier lorsque des modèles non natifs (encodeur-décodeur ou decoder-only) sont adaptés à des tâches de classification, de résumé ou d'explication.

Table des matières

1 Introduction **3**

2 Cadre expérimental et reproductibilité **3**

2.1 Exécution pratique et organisation du code 4

3 Exercice 1 : classification de clauses juridiques **4**

3.1 Question de recherche et stratégie de comparaison 4

3.2 Choix de modèles et formulation de tâche 5

3.3 Justification des paramètres et de l'optimisation 5

3.4 Résultats 6

3.5 Interprétation 6

4 Exercice 2 : résumé de documents juridiques **7**

4.1 Question de recherche et hypothèses 7

4.2 Justification des choix de modèles 7

4.3 Justification des paramètres et de l'optimisation 7

4.4 Résultats 8

4.5 Interprétation comparée 8

5 Exercice 3 : explication de clauses en anglais simple **9**
5.1 Question de recherche 9
5.2 Justification des choix expérimentaux 9
5.3 Justification des paramètres et de l'optimisation 9
5.4 Résultats 10
5.5 Exemples qualitatifs commentés (seed 42) 10
5.6 Interprétation et limites des decoder-only 15
5.7 Discussion conclusive (Exercice 3) 16

6 Discussion transversale **16**

7 Menaces à la validité **17**

8 Conclusion **17**

1 Introduction

Le sujet du TP demande explicitement de construire des *benchmarks* reproductibles pour trois tâches (classification, résumé, explication), avec des contraintes d'architecture et de métriques selon l'exercice. Pris à la lettre, cela n'impose pas toujours des comparaisons causales strictes entre modèles ; il serait possible de produire un benchmark minimal qui respecte uniquement la forme attendue.

Nous avons fait un choix plus exigeant : structurer les benchmarks de façon à rendre les résultats interprétables scientifiquement. Cette démarche n'est pas formulée de manière explicite dans l'énoncé, mais elle constitue, selon nous et après discussion sur le sujet avec Monsieur Dias (professeur de Traitement Automatique des Langues Avancées), une preuve de compréhension du TP : choisir des modèles qui peuvent être comparés *quand c'est possible*, expliciter quand la comparaison est forte ou seulement opérationnelle, puis relier les écarts observés à des hypothèses techniques défendables.

Concrètement, dans l'exercice 1, nous avons comparé trois adaptations sur un même backbone encodeur pour isoler l'effet de la méthode d'adaptation. Dans l'exercice 2, nous avons opposé une famille seq2seq (résumé natif) et une famille decoder-only (résumé non natif par prompting) pour étudier un compromis qualité/coût, mais aussi construit une comparaison intra-famille seq2seq entre modèles encodeur-décodeur : faut-il privilégier un modèle non distillé de taille moyenne, ou un modèle distillé issu d'un checkpoint initialement plus capacitaire ? Dans l'exercice 3, nous avons maintenu constant le backbone decoder-only et varié la stratégie de prompting, afin de mesurer l'effet du contrôle instructionnel sur la fidélité et la sûreté.

Au-delà des scores, la question centrale est la suivante : *quelles décisions expérimentales rendent les conclusions défendables dans un rapport scientifique, sous une contrainte matérielle réelle ?*

2 Cadre expérimental et reproductibilité

Toutes les expériences ont été exécutées sur l'environnement vdigpu de l'université, avec une contrainte de mémoire GPU d'environ 6 Go (A40-6Q). Cette contrainte a structuré l'ensemble des choix techniques : usage massif de LoRA/PEFT, précision mixte FP16 quand disponible, accumulation de gradient, troncature des séquences, checkpoints intermédiaires et reprise de run.

Un second facteur contraignant, discuté avec Monsieur Dias et signalé à la DSI, a également influencé notre protocole : au-delà du bridage de performance attendu, nous avons observé un bridage additionnel apparemment arbitraire et d'origine inconnue. Sur certaines exécutions, l'apprentissage, l'évaluation ou la génération se figent après une vingtaine de minutes, sans crash explicite ; le calcul semble toujours actif, mais le temps restant estimé peut exploser vers plusieurs centaines, voire milliers d'heures, sans processus externe clairement identifiable. Dans l'attente de l'investigation DSI, nous avons intégré cette contrainte opérationnelle dans nos choix d'implémentation : checkpoints fréquents, mécanismes de reprise, et limitation volontaire de l'exploration des modèles les plus volumineux.

Le protocole statistique est identique dans les trois exercices : graines fixes `42`, `123`, `456`, exécution triple de chaque configuration, agrégation en moyenne et écart-type dans des fichiers JSON finaux. Cette uniformité est essentielle car elle transforme un résultat ponctuel en tendance

robuste.

Sur les données, deux corpus sont utilisés : LEDGAR (clauses juridiques) pour les exercices 1 et 3, et BillSum (projets de loi + résumés humains) pour l'exercice 2. Dans tous les cas, le split test est tenu constant entre modèles à l'intérieur de chaque exercice.

2.1 Exécution pratique et organisation du code

Le dépôt est structuré par exercice (`ex1/`, `ex2/`, `ex3/`). Pour le rapport final, nous ne retenons que les scripts effectivement utilisés pour les benchmarks principaux ; les fichiers auxiliaires de génération de tableaux, les dossiers `other/` et les variantes exploratoires hors protocole final ne sont pas considérés ici.

Le mode `-debug` permet de vérifier rapidement qu'un pipeline s'exécute correctement sur un sous-ensemble réduit avant de lancer l'expérience complète. Un lancement sans `-debug` correspond à l'exécution standard du benchmark.

Exercice 1 – classification

```
python3 ex1/bert_full_finetuning.py
python3 ex1/bert_full_finetuning.py --debug
```

Exercice 2 – résumé

```
python3 ex2/t5_summarization_lora.py
python3 ex2/t5_summarization_lora.py --debug
```

Exercice 3 – explication

```
python3 ex3/distilgpt2_zeroshot_prompting.py
python3 ex3/distilgpt2_zeroshot_prompting.py --debug
```

Les sorties agrégées sont écrites dans des fichiers JSON du type `benchmark_results/<modele>_partial.jsonl` puis `benchmark_results/<modele>_final.json`. Pour l'exercice 3, les générations textuelles sont stockées dans `ex3_results/`. En pratique, nous avons toujours validé d'abord un run `-debug`, puis lancé la configuration complète utilisée dans le benchmark.

3 Exercice 1 : classification de clauses juridiques

3.1 Question de recherche et stratégie de comparaison

La question de fond de l'exercice 1 est de savoir comment adapter efficacement des architectures hétérogènes à une classification multi-classes difficile (100 labels), sous contrainte GPU. Le point méthodologique important est que nous n'avons pas choisi "trois encodeurs différents" au hasard, mais trois *adaptations* du même encodeur `bert-base-uncased` : full fine-tuning, linear probing strict, et LoRA. Ce choix permet de comparer proprement l'effet de la méthode d'adaptation.

Nous avons ensuite ajouté `t5-small` (encodeur-décodeur) et `gpt2` (decoder-only), non pas pour faire un concours direct de scores absolus entre architectures incomparables, mais pour tester explicitement la capacité de ces architectures à s'adapter à une tâche qui n'est pas leur usage natif, et documenter les compromis pratiques induits par cette adaptation.

3.2 Choix de modèles et formulation de tâche

Le BERT full fine-tuning représente la borne "performance potentielle" côté encodeur, au prix d'un coût d'optimisation plus élevé. Le linear probing représente la borne "adaptation minimale", utile pour quantifier ce qu'un backbone pré-entraîné capture sans adaptation profonde. LoRA sert de compromis : on adapte le modèle sans mettre à jour tous les poids.

Pour T5, la classification est reformulée en text-to-text : le modèle génère un label textualisé ensuite remappé vers l'id de classe. Nous n'avons pas ajouté de stratégie d'optimisation lourde sur `t5-small`, car il s'agit déjà d'un modèle compact et le gain attendu d'une adaptation supplémentaire était limité au regard du coût expérimental sous contrainte vdigpu. Pour GPT-2, la classification passe par une tête dédiée de classification de séquence, avec adaptation LoRA : le modèle restant plus conséquent, ce choix permet de réduire le coût d'entraînement tout en conservant un bon niveau de performance. Ce compromis était cohérent avec les observations faites sur les encodeurs dans l'exercice 1, et a aussi été confirmé sur le cas decoder-only après essais de plusieurs variantes d'optimisation. Ces deux cas ont été explicitement documentés car la tâche n'est pas native pour ces architectures.

3.3 Justification des paramètres et de l'optimisation

Pour les trois variantes BERT, nous avons volontairement conservé un socle de réglages commun afin de rendre la comparaison lisible : troncature à `128` tokens, `batch size` d'entraînement `4`, `batch size` d'évaluation `16`, accumulation de gradient `2` (soit un batch effectif de `8`), `2` époques, évaluation et sauvegarde à chaque époque, chargement du meilleur checkpoint en fin d'entraînement sur la base de la `eval_loss`, et `fp16` dès que le GPU le permet. Ces choix répondent directement à la contrainte vdigpu : `128` tokens suffisent pour une grande partie des clauses LEDGAR tout en gardant un débit acceptable, et le batch effectif de `8` stabilise l'optimisation sans dépasser la mémoire disponible.

La différence essentielle porte donc sur les hyperparamètres d'adaptation. Pour le full fine-tuning, nous avons retenu un taux d'apprentissage conservateur de `2e-5` avec `weight decay=0.01`, ce qui correspond à un réglage prudent pour mettre à jour tous les poids d'un encodeur pré-entraîné. Pour le linear probing strict, le backbone étant gelé et seule la tête étant apprise, nous avons augmenté le taux d'apprentissage à `2e-3`, supprimé la décroissance de poids (`0.0`) et réduit l'early stopping à une patience de `1`, car la convergence de la tête seule est beaucoup plus rapide. Pour LoRA sur BERT, nous avons utilisé `learning rate=2e-4`, `weight decay=0.01`, une configuration `r=8`, `alpha=16`, `dropout=0.1`, appliquée aux modules `query`, `key` et `value` ; ce réglage matérialise le compromis attendu entre capacité d'adaptation et coût mémoire.

Les deux modèles non natifs ont reçu des réglages cohérents avec leur formulation. Pour `t5-small`, nous avons utilisé `max input=256`, `max target=32`, `generated label max length=32`, `batch size=8`, 3 époques, `learning rate=5e-5`, `weight decay=0.01`, avec évaluation/sauvegarde tous les `500` pas et early stopping de patience `3` ; l'objectif était de laisser au modèle seq2seq assez de marge pour apprendre la projection clause → label textuel sans surcoût excessif. Pour `gpt2`, nous avons retenu `max length=256`, `batch size=4`, accumulation `4` (batch effectif `16`), `2` époques, `learning rate=2e-4`, `weight decay=0.01`, évaluation/sauvegarde tous les `500` pas, early stopping de patience `3`, et LoRA sur `c_attn`, `c_proj` et `c_fc` ; ici encore, le choix vise à

compenser le coût d'un decoder-only en maintenant une adaptation paramétriquement efficace.

3.4 Résultats

Table 1 – Exercice 1 – benchmark LEDGAR (moyenne ± écart-type, 3 runs)

Modèle	Architecture	Adaptation	Accuracy	F1-macro	AUROC
`bert-base-uncased-full-fast`	Encodeur	Full fine-tuning	0.8498 ± 0.0015	0.7439 ± 0.0032	0.9891 ± 0.0005
`bert-base-uncased-linear-probing-full-fast`	Encodeur	Linear probing strict	0.5159 ± 0.0024	0.3332 ± 0.0031	0.9436 ± 0.0008
`bert-base-uncased-lora-full-fast`	Encodeur	LoRA + classifier	0.7997 ± 0.0022	0.6555 ± 0.0029	0.9850 ± 0.0004
`t5-small`	Encodeur-Decodeur	Text-to-text	0.8267 ± 0.0004	0.7339 ± 0.0035	0.8642 ± 0.0021
`gpt2`	Decodeur-only	LoRA + cls head	0.8405 ± 0.0014	0.7532 ± 0.0034	0.9935 ± 0.0003

3.5 Interprétation

Dans ce sous-benchmark, la comparaison directe est méthodologiquement légitime, car les trois configurations reposent sur le même backbone (`bert-base-uncased`) et le même protocole expérimental ; seule la stratégie d'adaptation change.

Pour `bert-base-uncased-full-fast`, le fine-tuning complet constitue la borne haute observée (accuracy 0.8498, F1-macro 0.7439, AUROC 0.9891). Les trois métriques convergent vers une même lecture : le modèle parvient à ajuster ses représentations internes de manière suffisamment fine pour une classification juridique à 100 classes.

Pour `bert-base-uncased-linear-probing-full-fast`, l'adaptation minimale dégrade fortement la décision multiclasses (accuracy 0.5159, F1-macro 0.3332), alors que l'AUROC reste plus élevé (0.9436). Cette combinaison suggère que le backbone conserve une information de rang utile, mais que l'absence d'adaptation profonde empêche de transformer cette information en frontières de décision robustes entre classes proches.

Pour `bert-base-uncased-lora-full-fast`, l'adaptation paramétriquement efficace rétablit une large partie du niveau full fine-tuning (accuracy 0.7997, F1-macro 0.6555, AUROC 0.9850). L'accuracy et le F1-macro montrent que la capacité de décision est nettement restaurée, et l'AUROC indique que la qualité de séparation probabiliste reste proche de la borne haute.

En synthèse comparative, l'écart full vs linear est massif (+0.3339 en accuracy ; +0.4107 en F1-macro), tandis que LoRA réduit fortement ce décrochage (écart full vs LoRA limité à 0.0501 en accuracy et 0.0884 en F1-macro). Dans notre contexte vdigpu, cela valide précisément l'intérêt méthodologique de LoRA : un compromis robuste entre coût d'adaptation et qualité de décision multiclasses.

Dans ce rapport, nous ne traitons pas T5 et GPT-2 comme des candidats directement comparables au sens strict, car ils relèvent de familles architecturales différentes et de mécanismes de décision différents. L'analyse est donc conduite séparément, en évaluant pour chaque cas la qualité de l'adaptation d'une architecture non native à la tâche de classification.

Pour `t5-small`, la classification par génération de label puis remappage reste viable dans notre protocole (accuracy 0.8267, F1-macro 0.7339, AUROC 0.8642). L'accuracy montre qu'une part majoritaire des décisions est correcte au niveau global ; le F1-macro indique que la performance reste plus hétérogène entre classes et que les classes difficiles ne sont pas toutes captées au même niveau ; l'AUROC confirme enfin une séparabilité probabiliste utilisable malgré le caractère non natif de la formulation. Cette lecture permet de juger la capacité d'un encodeur-décodeur à se

conformer à une tâche de décision multiclasses qui n'est pas son usage natif.

Pour `gpt2`, l'adaptation par tête de classification avec LoRA est également viable (accuracy 0.8405, F1-macro 0.7532, AUROC 0.9935). L'accuracy confirme une bonne qualité de décision globale ; le F1-macro, plus exigeant sur l'équilibre inter-classes, montre que la performance reste solide sur la distribution multiclasses ; l'AUROC indique une très bonne hiérarchisation probabiliste dans ce protocole. Ici, l'enjeu est d'évaluer la capacité d'un decoder-only à stabiliser une décision de classe sous contrainte d'adaptation paramétriquement efficace.

En synthèse, les deux adaptations fonctionnent et produisent des résultats exploitables. On peut signaler, avec prudence, que dans notre protocole GPT-2 obtient des scores plus élevés sur les métriques reportées ; néanmoins, la conclusion principale reste que chaque architecture peut être adaptée de façon crédible à la tâche, avec des compromis méthodologiques propres.

4 Exercice 2 : résumé de documents juridiques

4.1 Question de recherche et hypothèses

L'exercice 2 cherche à quantifier une tension classique : faut-il privilégier des architectures nativement seq2seq pour le résumé, ou des decoder-only promptés qui peuvent être plus flexibles et parfois plus économes ? Nous avons formulé deux hypothèses :

1. à protocole d'optimisation comparable, les modèles seq2seq devraient mieux couvrir le contenu de référence ;
2. la distillation côté decoder-only devrait améliorer le coût, avec un risque de baisse modérée des métriques ;
3. au sein des seq2seq, un modèle distillé issu d'un checkpoint historiquement plus capacitaire peut dépasser un modèle non distillé de taille moyenne.

4.2 Justification des choix de modèles

Nous avons choisi trois modèles de la famille encodeur-décodeur (`t5-small`, `bart-base`, `distilbart-cnn-12-6`) et deux decoder-only (`gpt2`, `distilgpt2`). Ce choix est méthodologiquement utile pour séparer les effets d'architecture et de formulation, tout en permettant des comparaisons intra-famille rigoureuses.

Dans la famille seq2seq, garder une logique LoRA homogène permet d'éviter qu'un modèle soit artificiellement favorisé par un tuning ad hoc plus agressif. Dans la famille decoder-only, comparer GPT-2 et son distillé permet de tester un compromis capacité/rapidité dans un cadre expérimental contraint et reproductible.

La formulation decoder-only a été explicitement traitée : prompt causal et masquage de la perte sur la partie instruction, de sorte que l'optimisation cible uniquement le résumé généré.

4.3 Justification des paramètres et de l'optimisation

Nous avons ici cherché un protocole homogène par famille plutôt qu'un réglage maximalement spécialisé pour chaque checkpoint. Pour les trois modèles seq2seq (`t5-small`, `bart-base`, `distilbart-cnn-12-6`), nous avons utilisé `max source length=512`, `max target length=128`,

batch size train=2, batch size eval=4, accumulation de gradient 8 (batch effectif 16), 2 époques, learning rate=1e-4, weight decay=0.01, warmup ratio=0.05, scheduler cosinus, sauvegarde par époque, sans sélection finale sur une validation intermédiaire (evaluation_strategy="no", load_best_model_at_end=False). Le choix est principalement dicté par la mémoire disponible : 512 tokens conservent une part utile du texte législatif, tandis que le batch effectif 16 évite une optimisation trop bruitée malgré un micro-batch faible.

La couche LoRA a été gardée volontairement simple et constante : r=8, alpha=16, dropout=0.1. Les modules ciblés suivent l'architecture : q/k/v/o pour T5, q_proj/k_proj/v_proj/out_proj pour BART et DistilBART. Ce choix permet de concentrer l'adaptation sur les projections d'attention, ce qui est cohérent avec une tâche de résumé conditionnel, tout en limitant fortement le nombre de paramètres appris. À la génération, nous avons fixé max_new_tokens=96 ; pour les seq2seq, nous avons en plus retenu num_beams=4 afin d'améliorer la stabilité du recouvrement de contenu sans faire exploser le coût.

Pour les deux decoder-only (gpt2, distilgpt2), nous avons gardé la même philosophie mais avec une formulation compatible causal : max source length=320, max target length=96, max total length=448, batch size train=2, batch size eval=2, accumulation 8, 2 époques, learning rate=2e-4, weight decay=0.01, warmup ratio=0.05, scheduler cosinus, LoRA r=8, alpha=16, dropout=0.1 sur c_attn et c_proj. Le taux d'apprentissage un peu plus élevé est ici défendable, car seuls des sous-espaces LoRA sont adaptés. En génération, nous avons retenu max_new_tokens=96 et num_beams=1 : pour ces modèles, une recherche plus coûteuse aurait fortement pénalisé le runtime sous vdigpu sans garantie d'amélioration qualitative proportionnée.

4.4 Résultats

Table 2 – Exercice 2 – benchmark BillSum (moyenne ± écart-type, 3 runs)

Modèle	Architecture	Adaptation	BLEU	ROUGE-1	ROUGE-2
t5-small-sum-lora-fast	Encodeur-Decodeur	LoRA seq2seq	7.1962 ± 0.0427	36.5806 ± 0.0550	21.2486 ± 0.0248
bart-base-sum-lora-fast	Encodeur-Decodeur	LoRA seq2seq	5.5130 ± 0.3490	36.9285 ± 0.3389	23.0238 ± 0.0538
distilbart-cnn-sum-lora-fast	Encodeur-Decodeur	LoRA seq2seq	8.9973 ± 0.0292	40.6954 ± 0.0201	24.7623 ± 0.0582
gpt2-sum-lora-fast	Decodeur-only	Prompt causal + LoRA	11.9219 ± 0.0140	34.0117 ± 0.0411	15.9570 ± 0.0269
distilgpt2-sum-lora-fast	Decodeur-only	Prompt causal + LoRA	11.5507 ± 0.0170	33.4500 ± 0.0580	15.4346 ± 0.0200

4.5 Interprétation comparée

Comme pour l'exercice 1, nous distinguons ici les comparaisons rigoureuses (intra-famille) des lectures seulement indicatives (inter-familles). Les comparaisons strictes sont menées entre modèles de même famille et de protocole homogène ; à l'inverse, la lecture seq2seq vs decoder-only reste une observation d'ensemble utile, mais non causale au sens strict, car les mécanismes de génération et la formulation de tâche diffèrent.

Dans la famille seq2seq, la lecture multicritère (BLEU, ROUGE-1, ROUGE-2) est cohérente et relativement stable : distilbart-cnn-sum-lora-fast obtient le meilleur niveau global (BLEU 8.9973, ROUGE-1 40.6954, ROUGE-2 24.7623), devant bart-base-sum-lora-fast (5.5130 ; 36.9285 ; 23.0238) et t5-small-sum-lora-fast (7.1962 ; 36.5806 ; 21.2486). Ce résultat répond directement à notre question intra-famille : dans ce protocole, un modèle distillé issu d'un checkpoint historiquement plus capacitaire et spécialisé résumé (distilbart-cnn-12-6) peut dépasser un modèle

non distillé de taille moyenne (`bart-base`), y compris sur les métriques de couverture structurelle (`ROUGE-2`).

`t5-small` reste néanmoins une alternative intéressante : il dépasse `bart-base` sur BLEU (7.1962 vs 5.5130) et reste proche en ROUGE-1 (36.5806 vs 36.9285), avec un backbone plus compact et générique. En pratique, il peut donc constituer un choix pertinent lorsque l'on privilégie la simplicité d'adaptation et un compromis qualité/coût robuste, même s'il demeure derrière sur ROUGE-2 dans ce protocole.

Dans la famille decoder-only, la comparaison `gpt2` vs `distilgpt2` est également directe : le parent conserve l'avantage en qualité (BLEU 11.9219 vs 11.5507 ; ROUGE-1 34.0117 vs 33.4500 ; ROUGE-2 15.9570 vs 15.4346). La conclusion d'ingénierie reste classique : si la priorité est la qualité de résumé, le parent est préférable ; si la priorité est le coût/temps d'itération, la version distillée conserve sa légitimité pratique.

Enfin, la lecture inter-familles doit être formulée prudemment : les decoder-only apparaissent forts sur `BLEU`, mais plus faibles sur `ROUGE-2`, alors que les seq2seq gardent un avantage net sur les métriques de recouvrement de contenu et de structure. Cette dissociation est informative pour la formulation de tâche : elle suggère qu'en résumé juridique, la proximité lexicale locale ne suffit pas si la structuration informationnelle attendue par la référence est la priorité principale.

5 Exercice 3 : explication de clauses en anglais simple

5.1 Question de recherche

L'exercice 3 déplace la difficulté : il n'y a pas de référence unique, et la qualité dépend simultanément de la fidélité juridique, de la lisibilité pour non-experts et de la sûreté (ne pas introduire d'obligations absentes). La question de recherche n'est donc pas "quel score unique est maximal ?", mais "quel niveau de contrôle obtient-on, à modèle constant, en faisant varier uniquement la stratégie de prompting decoder-only ?"

5.2 Justification des choix expérimentaux

Nous avons volontairement conservé le même backbone decoder-only (`distilbert/distilgpt2`) et varié la stratégie de prompting : zero-shot direct, roleplay avec contraintes de sûreté, et few-shot ICL. Cette conception rend la comparaison intra-exercice méthodologiquement forte : on contrôle l'architecture, les données et l'inférence, et l'on isole principalement l'effet du prompt.

Les trois configurations partagent le même split test LEDGAR (10 000 clauses), les mêmes seeds, et les mêmes hyperparamètres principaux de génération (`max_input_tokens=256`, `max_new_tokens=120`, `temperature=0.2`, `top_p=0.9`). Les différences observées peuvent donc être attribuées majoritairement à la stratégie de prompt, sous réserve des limites usuelles de variance générationnelle.

5.3 Justification des paramètres et de l'optimisation

L'exercice 3 ne repose pas sur un fine-tuning supplémentaire mais sur un protocole d'inférence contrôlée ; la discussion porte donc ici sur des hyperparamètres de génération plutôt que sur des paramètres d'entraînement. Nous avons fixé, pour les trois stratégies, `batch size=8`, `max_input_tokens=256`, `max_new_tokens=120`, `temperature=0.2` et `top_p=0.9`, avec exécution

en `fp16` quand le GPU le permet. Le choix de `256` tokens répond à un compromis simple : conserver une partie substantielle de la clause tout en évitant que le coût d'inférence explose sur 10 000 exemples ; le plafond à `120` tokens force par ailleurs des explications relativement courtes, ce qui est cohérent avec une cible non experte et limite les dérives répétitives.

Le couple `temperature=0.2` / `top_p=0.9` a été choisi pour réduire la variance sans imposer une génération totalement déterministe. Dans une tâche juridique sensible, nous voulions éviter une créativité excessive, mais conserver un minimum de souplesse lexicale pour permettre la reformulation. Le batch `8` représente le meilleur compromis observé entre débit et stabilité mémoire sur vdigpu ; les scripts réduisent d'ailleurs automatiquement le batch en cas d'erreur CUDA, ce qui traduit bien que ce réglage était proche de la limite opérationnelle utile.

La seule différence intentionnelle entre configurations est donc la structure du prompt. En `zero-shot-direct`, la consigne est minimale pour maximiser le débit et limiter la compétition de contexte. En `roleplay-safety`, nous ajoutons explicitement un rôle système et des contraintes de sûreté pour tester si un contrôle instructionnel plus fort améliore la fidélité. En `few-shot-icl`, nous injectons deux démonstrations fixes, identiques sur tous les runs, afin de tester un apprentissage en contexte sans changer ni modèle ni décodage. Cette discipline expérimentale est importante : elle permet d'attribuer les écarts observés non à une différence de backbone ou de réglage de sampling, mais bien à la stratégie de prompting elle-même.

5.4 Résultats

Table 3 – Exercice 3 – qualité/sûreté (moyenne ± écart-type, 3 runs)

Stratégie	**Coverage %**	**Novel %**	**Added obligation %**	**Seed consistency %**
zero-shot-direct	73.81 ± 0.02	30.20 ± 0.07	9.56 ± 0.16	66.52 ± 0.00
roleplay-safety	72.28 ± 0.06	35.93 ± 0.07	5.26 ± 0.25	66.51 ± 0.00
few-shot-icl	47.76 ± 0.06	43.67 ± 0.19	26.23 ± 0.32	56.38 ± 0.00

Table 4 – Exercice 3 – coût de génération

Stratégie	**Runtime (s)**	**Samples/s**	**Avg generated words**
zero-shot-direct	593.01 ± 14.05	16.87 ± 0.39	142.42 ± 0.13
roleplay-safety	599.80 ± 4.54	16.67 ± 0.13	145.15 ± 0.03
few-shot-icl	605.54 ± 9.95	16.52 ± 0.27	108.87 ± 0.07

5.5 Exemples qualitatifs commentés (seed 42)

Les exemples ci-dessous sont reportés en version *complète* (clause, prompt et sortie brute) pour rendre l'analyse qualitative vérifiable. Pour une comparaison stricte, les trois exemples utilisent la **même clause source** (seed 42, même enregistrement), seule la stratégie de prompting change.

Exemple 1 – zero-shot-direct (cas limite)

Contexte : label `Remedies` (breach/threatened breach, forfeiture et injonction).

Clause d'origine (complète)

```
Participant agrees that in the event of a breach or threatened breach of any of the covenants
↪ contained in Sections 10 or 11 of this Agreement, in addition to any other penalties or
↪ restrictions that may apply under any employment agreement, state law, or otherwise,
↪ Participant shall forfeit, upon written notice to such effect from the Corporation, any and
↪ all Awards granted to him or her under the Plan and this Agreement, including vested Awards.
↪ The forfeiture provisions of this Section 13 shall continue to apply, in accordance with
↪ their terms, after the provisions of any employment or other agreement between the
↪ Corporation and Participant have lapsed. Participant consents and agrees that if Participant
↪ violates or threatens to violate any provisions of Sections 10 or 11 of this Agreement, the
↪ Corporation or its successors in interest shall be entitled, in addition to any other
↪ remedies that they may have, including money damages, to an injunction to be issued by a
↪ court of competent jurisdiction restraining Participant from committing or continuing any
↪ violation of Sections 10 or 11. In the event that Participant is found to have breached any
↪ provision set forth in Section 10 of this Agreement, the time period provided for in that
↪ provision shall be deemed tolled (i.e., it will not begin to run) for as long as Participant
↪ was in violation of that provision. The provisions of Sections 10 and 11 of this Agreement
↪ shall continue to apply, in accordance with their terms, after the Participant's service has
↪ terminated and after provisions of any employment or other agreement between the Corporation
↪ and the Participant have lapsed.
```

Prompt complet utilisé

```
Explain the following legal clause in plain English for a non-expert.
Do not add obligations that are not in the clause.

Clause:
Participant agrees that in the event of a breach or threatened breach of any of the covenants
↪ contained in Sections 10 or 11 of this Agreement, in addition to any other penalties or
↪ restrictions that may apply under any employment agreement, state law, or otherwise,
↪ Participant shall forfeit, upon written notice to such effect from the Corporation, any and
↪ all Awards granted to him or her under the Plan and this Agreement, including vested Awards.
↪ The forfeiture provisions of this Section 13 shall continue to apply, in accordance with
↪ their terms, after the provisions of any employment or other agreement between the
↪ Corporation and Participant have lapsed. Participant consents and agrees that if Participant
↪ violates or threatens to violate any provisions of Sections 10 or 11 of this Agreement, the
↪ Corporation or its successors in interest shall be entitled, in addition to any other
↪ remedies that they may have, including money damages, to an injunction to be issued by a
↪ court of competent jurisdiction restraining Participant from committing or continuing any
↪ violation of Sections 10 or 11. In the event that Participant is found to have breached any
↪ provision set forth in Section 10 of this Agreement, the time period provided for in that
↪ provision shall be deemed tolled (i.e., it will not begin to run) for as long as Participant
↪ was in violation of that provision. The provisions of Sections 10 and 11 of this Agreement
↪ shall continue to apply, in accordance with their terms, after the Participant's service has
↪ terminated and after provisions of any employment or other agreement between the Corporation
↪ and the Participant have lapsed.

Plain-English explanation:
```

Sortie complète générée

```
13, the Corporation or its successors in interest shall be entitled to a refund of any amount or
↪ other compensation that may be paid to Participant. Participant agrees that in the event of a
↪ breach or threatened breach of any of the covenants contained in Section 13, the Corporation
↪ or its successors in interest shall be entitled to a refund of any amount or other
↪ compensation that may be paid to Participant. Participant agrees that in the event of a
↪ breach or threatened breach of any of the covenants contained in Section 13, the Corporation
↪ or its successors in interest shall be entitled to a refund of any amount
```

Commentaire : sortie partiellement ancrée dans la clause mais avec dérive répétitive et simplification limitée ; le modèle reconstruit mal la structure complète des conditions et sanctions.

Exemple 2 – roleplay-safety (cas utile mais imparfait)

Contexte : même clause, même label `Remedies`.

Clause d'origine (complète)

```
Participant agrees that in the event of a breach or threatened breach of any of the covenants
↪ contained in Sections 10 or 11 of this Agreement, in addition to any other penalties or
↪ restrictions that may apply under any employment agreement, state law, or otherwise,
↪ Participant shall forfeit, upon written notice to such effect from the Corporation, any and
↪ all Awards granted to him or her under the Plan and this Agreement, including vested Awards.
↪ The forfeiture provisions of this Section 13 shall continue to apply, in accordance with
↪ their terms, after the provisions of any employment or other agreement between the
↪ Corporation and Participant have lapsed. Participant consents and agrees that if Participant
↪ violates or threatens to violate any provisions of Sections 10 or 11 of this Agreement, the
↪ Corporation or its successors in interest shall be entitled, in addition to any other
↪ remedies that they may have, including money damages, to an injunction to be issued by a
↪ court of competent jurisdiction restraining Participant from committing or continuing any
↪ violation of Sections 10 or 11. In the event that Participant is found to have breached any
↪ provision set forth in Section 10 of this Agreement, the time period provided for in that
↪ provision shall be deemed tolled (i.e., it will not begin to run) for as long as Participant
↪ was in violation of that provision. The provisions of Sections 10 and 11 of this Agreement
↪ shall continue to apply, in accordance with their terms, after the Participant's service has
↪ terminated and after provisions of any employment or other agreement between the Corporation
↪ and the Participant have lapsed.
```

Prompt complet utilisé

```
System role: You are a legal simplification assistant.
Task: explain legal clauses for non-lawyers in plain English.
Hard constraints: do not provide legal advice, do not invent obligations, keep factual tone.

User clause:
```

```
Participant agrees that in the event of a breach or threatened breach of any of the covenants
↪ contained in Sections 10 or 11 of this Agreement, in addition to any other penalties or
↪ restrictions that may apply under any employment agreement, state law, or otherwise,
↪ Participant shall forfeit, upon written notice to such effect from the Corporation, any and
↪ all Awards granted to him or her under the Plan and this Agreement, including vested Awards.
↪ The forfeiture provisions of this Section 13 shall continue to apply, in accordance with
↪ their terms, after the provisions of any employment or other agreement between the
↪ Corporation and Participant have lapsed. Participant consents and agrees that if Participant
↪ violates or threatens to violate any provisions of Sections 10 or 11 of this Agreement, the
↪ Corporation or its successors in interest shall be entitled, in addition to any other
↪ remedies that they may have, including money damages, to an injunction to be issued by a
↪ court of competent jurisdiction restraining Participant from committing or continuing any
↪ violation of Sections 10 or 11. In the event that Participant is found to have breached any
↪ provision set forth in Section 10 of this Agreement, the time period provided for in that
↪ provision shall be deemed tolled ( i.e. , it will not begin to run) for as long as
↪ Participant was in violation of that provision. The provisions of Sections 10 and 11 of this
↪ Agreement shall continue to apply, in accordance with their terms, after the Participant's
↪ service has terminated and after provisions of any employment or other agreement between the
↪ Corporation and the Participant have lapsed.

Assistant explanation:
```

Sortie complète générée

```
Participant agrees that the provisions of Section 13 shall continue to apply, in accordance with
↪ their terms, after the provisions of any employment or other agreement between the
↪ Corporation and Participant have lapsed. Participant agrees that if Participant violates or
↪ threatens to violate any provisions of Sections 10 or 11 of this Agreement, the Corporation
↪ or its successors in interest shall be entitled, in addition to any other remedies that they
↪ may have, including money damages, to an injunction to be issued by a court of competent
↪ jurisdiction restraining Participant from committing or continuing any violation of Sections
↪ 10 or 11 of this Agreement, in addition to any
```

Commentaire : sortie plus fidèle que le zero-shot sur les éléments centraux (Section 13, injonction), sans ajout explicite de conseil juridique ; la vulgarisation reste toutefois incomplète et tronquée.

Exemple 3 – few-shot-icl (contamination explicite)

Contexte : même clause, même label `Remedies`.

Clause d'origine (complète)

```
Participant agrees that in the event of a breach or threatened breach of any of the covenants
↪ contained in Sections 10 or 11 of this Agreement, in addition to any other penalties or
↪ restrictions that may apply under any employment agreement, state law, or otherwise,
↪ Participant shall forfeit, upon written notice to such effect from the Corporation, any and
↪ all Awards granted to him or her under the Plan and this Agreement, including vested Awards.
↪ The forfeiture provisions of this Section 13 shall continue to apply, in accordance with
↪ their terms, after the provisions of any employment or other agreement between the
↪ Corporation and Participant have lapsed. Participant consents and agrees that if Participant
↪ violates or threatens to violate any provisions of Sections 10 or 11 of this Agreement, the
↪ Corporation or its successors in interest shall be entitled, in addition to any other
↪ remedies that they may have, including money damages, to an injunction to be issued by a
↪ court of competent jurisdiction restraining Participant from committing or continuing any
↪ violation of Sections 10 or 11. In the event that Participant is found to have breached any
↪ provision set forth in Section 10 of this Agreement, the time period provided for in that
↪ provision shall be deemed tolled (i.e., it will not begin to run) for as long as Participant
↪ was in violation of that provision. The provisions of Sections 10 and 11 of this Agreement
↪ shall continue to apply, in accordance with their terms, after the Participant's service has
↪ terminated and after provisions of any employment or other agreement between the Corporation
↪ and the Participant have lapsed.
```

Prompt complet utilisé

```
You explain legal clauses in plain English for non-experts.
Stay faithful to the clause. Do not add legal advice or new obligations.

Example 1
Clause: This Agreement shall be governed by and construed in accordance with the laws of the
↪ State of New York, without regard to conflict of laws principles.
Explanation: If there is a legal dispute about this contract, New York law will be used to
↪ interpret it.

Example 2
Clause: Either party may terminate this Agreement upon thirty (30) days prior written notice to
↪ the other party.
Explanation: Either side can end the contract, but they must send written notice at least 30 days
↪ in advance.

Now explain the next clause.
```

```
Clause: Participant agrees that in the event of a breach or threatened breach of any of the
↪ covenants contained in Sections 10 or 11 of this Agreement, in addition to any other
↪ penalties or restrictions that may apply under any employment agreement, state law, or
↪ otherwise, Participant shall forfeit, upon written notice to such effect from the
↪ Corporation, any and all Awards granted to him or her under the Plan and this Agreement,
↪ including vested Awards. The forfeiture provisions of this Section 13 shall continue to
↪ apply, in accordance with their terms, after the provisions of any employment or other
↪ agreement between the Corporation and Participant have lapsed. Participant consents and
↪ agrees that if Participant violates or threatens to violate any provisions of Sections 10 or
↪ 11 of this Agreement, the Corporation or its successors in interest shall be entitled, in
↪ addition to any other remedies that they may have, including money damages, to an injunction
↪ to be issued by a court of competent jurisdiction restraining Participant from committing or
↪ continuing any violation of Sections 10 or 11. In the event that Participant is found to have
↪ breached any provision set forth in Section 10 of this Agreement, the time period provided
↪ for in that provision shall be deemed tolled (i.e., it will not begin to run) for as long as
↪ Participant was in violation of that provision. The provisions of Sections 10 and 11 of this
↪ Agreement shall continue to apply, in accordance with their terms, after the Participant's
↪ service has terminated and after provisions of any employment or other agreement between the
↪ Corporation and the Participant have lapsed.
Explanation:
```

Sortie complète générée

```
with the laws of the State of New York, without regard to conflict of laws principles. Example 3
↪ Clause: This Agreement shall be governed by and construed in accordance with the laws of the
↪ State of New York, without regard to conflict of laws principles. Explanation: If there is a
↪ legal dispute about this contract, New York law will be used to interpret it. Example 4
↪ Clause: If there is a legal dispute about this contract, New York law will be used to
↪ interpret it. Explanation: If there is a legal dispute about this contract,
```

Commentaire : contamination forte par les démonstrations few-shot (thème “New York law”), perte de fidélité à la clause cible et fuite explicite de format.

5.6 Interprétation et limites des decoder-only

La comparaison est ici directement interprétable, car les trois stratégies partagent le même modèle et le même protocole d’inférence. Sur l’exemple qualitatif contrôlé (même clause), les comportements sont clairement différenciés : `zero-shot-direct` dérive vers des répétitions partielles, `roleplay-safety` reste le plus proche du contenu juridique utile (sections et remèdes), et `few-shot-icl` présente une contamination explicite par les démonstrations de prompt.

Cette lecture locale est cohérente avec les tendances globales agrégées sur 10 000 clauses : le `zero-shot-direct` garde le meilleur niveau de couverture brute et le meilleur débit (coverage 73.81, 16.87 samples/s), mais avec un risque d’ajout normatif plus élevé (added obligation 9.56). Le `roleplay-safety` réduit nettement ce risque (5.26) pour une baisse limitée de coverage (72.28) et un surcoût runtime modeste (599.80 s vs 593.01 s). Le `few-shot-icl` se dégrade sur les dimensions critiques (coverage 47.76, added obligation 26.23, seed consistency 56.38), ce qui confirme sa fragilité dans ce cadre contraint.

L’interprétation méthodologique devient donc plus précise : le prompt agit comme un levier de contrôle, mais ce levier n’est pas uniforme selon les clauses. Un gain moyen de sûreté (roleplay) n’implique pas une robustesse parfaite cas par cas ; inversement, un bon score de couverture moyen (zero-shot) n’assure pas une explication stable et fidèle sur chaque instance.

En synthèse d'ingénierie, `roleplay-safety` reste le compromis le plus prudent pour une utilisation sensible, `zero-shot-direct` peut être retenu lorsque la contrainte principale est le débit, et `few-shot-icl` n'est pas recommandé dans ce protocole. Plus généralement, les decoder-only restent très sensibles au contrôle instructionnel : sans garde-fous explicites (prompting robuste, post-vérification), la fidélité juridique et la sûreté restent difficiles à garantir de manière homogène.

5.7 Discussion conclusive (Exercice 3)

Les symptômes observés sont cohérents entre l'analyse quantitative et les exemples qualitatifs : (i) *echo de prompt* (le modèle recopie des segments d'instruction), (ii) *contamination de contexte* en few-shot (les exemples injectés prennent le dessus sur la clause cible), (iii) *dérive répétitive* (boucles lexicales), et (iv) *fidélité partielle* (certains éléments juridiques sont correctement repris, mais de manière incomplète ou mal structurée). Ces symptômes ne sont pas anecdotiques : ils signalent une limite structurelle du réglage decoder-only dans une tâche de transformation fidèle sous contrainte sémantique.

Le lien avec l'architecture est direct. Un decoder-only optimisé pour la prochaine unité de texte excelle en génération libre, mais n'impose pas naturellement une correspondance stricte *source* $\rightarrow$ *explication*. Quand la consigne devient longue (roleplay/few-shot), la compétition pour le budget contextuel augmente et la stabilité baisse ; quand la consigne est courte (zero-shot), le modèle reste plus rapide mais plus exposé aux reformulations fragiles et aux ajouts implicites. Autrement dit, dans ce cadre, la difficulté principale n'est pas seulement la "qualité de langue", mais le *contrôle de fidélité conditionnelle*.

Une piste de modèle plus adaptée est donc d'utiliser une architecture **encodeur-décodeur instruction-tuned** (par exemple FLAN-T5 ou un modèle T5/BART spécialisé simplification), éventuellement complétée par un contrôle explicite de sûreté (vérification NLI/entailment clause-explication, ou filtre post-génération). Ce type de modèle est généralement mieux aligné avec une tâche de reformulation contrainte : l'encodeur ancre la clause source, le décodeur produit la paraphrase, et l'ensemble réduit le risque de dérive purement auto-régressive observé ici.

6 Discussion transversale

Les trois exercices convergent vers un enseignement commun : il n'existe pas de "meilleur modèle" universel. Le bon choix dépend du triplet **architecture** – **formulation de tâche** – **contrainte système**.

Dans l'exercice 1, les encodeurs confirment leur avantage structurel pour la classification fine, et LoRA prouve qu'une adaptation efficace est possible sans full fine-tuning intégral. Dans l'exercice 2, la supériorité des seq2seq sur la couverture de contenu coexiste avec une compétitivité BLEU des decoder-only, ce qui rappelle l'importance d'une lecture multicritère. Dans l'exercice 3, la comparaison de prompts montre que la sûreté n'est pas un simple effet de taille du modèle : c'est un problème de contrôle de génération, fortement sensible à la formulation.

Cette lecture met l'accent sur l'interprétation des mécanismes observés, au-delà d'une simple exécution de scripts.

7 Menaces à la validité

Les limites majeures ont déjà été discutées dans les sections d'interprétation ; nous les synthétisons ici en version courte.

Premièrement, les comparaisons inter-familles (notamment en exo 2) restent principalement *opérationnelles* et non causales strictes : elles sont utiles pour une décision d'ingénierie, mais ne doivent pas être lues comme un effet d'architecture isolé.

Deuxièmement, en exo 3, le protocole isole volontairement l'effet du prompt à backbone constant ; les résultats portent donc d'abord sur des *stratégies de contrôle* et non sur une hiérarchie générale de modèles.

Troisièmement, les exemples qualitatifs reportés dans ce rapport correspondent à une inspection experimenteur de sorties (avec clause/prompt/réponse), ce qui améliore la traçabilité, mais ce n'est pas une campagne d'annotation juridique externe avec protocole inter-annotateurs.

Les extensions les plus directes sont : (i) annotation humaine experte (fidélité, lisibilité, risque), (ii) vérification sémantique post-génération (entailment/contradiction), (iii) comparaison decoder-only sur plusieurs backbones de taille voisine pour mieux séparer effet modèle et effet prompt.

8 Conclusion

Ce projet conduit à une conclusion simple mais importante : en TAL appliqué, la question pertinente n'est pas seulement "quel modèle marche le mieux ?", mais "dans quelles conditions une architecture devient-elle adaptée à une tâche donnée ?". Les trois exercices montrent en effet que la performance dépend moins d'un palmarès abstrait que de l'ajustement entre **architecture**, **formulation de tâche** et **contrainte système**. En classification, le contraste entre full fine-tuning, linear probing et LoRA montre que la profondeur d'adaptation compte autant que le backbone lui-même. En résumé, l'avantage des seq2seq sur la couverture structurelle confirme qu'une tâche de transformation conditionnelle favorise naturellement les architectures encodeur-décodeur. En explication de clauses, enfin, les limites observées chez les decoder-only montrent qu'une bonne génération libre ne suffit pas lorsqu'on exige simultanément fidélité, lisibilité et sûreté.

La contribution principale du rapport est donc d'avoir produit non seulement un benchmark reproductible, mais un benchmark *interprétable*. Sous contrainte vdigpu, nous n'avons pas cherché à maximiser des scores isolés : nous avons essayé de construire des comparaisons défendables, d'expliciter les choix d'optimisation, et de relier les écarts observés à des mécanismes plausibles. C'est, selon nous, ce qui donne à ce travail sa valeur scientifique : un bon benchmark ne se contente pas de classer des modèles, il aide à comprendre pourquoi ils réussissent, échouent, ou changent de compromis selon la tâche.